Aura Retail OS (Path – B)

Final Project Report

Course: OOPs

Project Link : github.com/Diffusity/Aura_Retail_OS

Team Name: Qties

Team Members :

Parth Maharaja (20212061)

Nevil Nandasana (202512009)

Vatsal Kamani (202512068)

Jaydip Chaudhari (202512090)

1. Problem Understanding :

1.1 Project Overview :

Aura Retail OS is a smart-city retail management platform developed for Zephyrus to operate autonomous kiosks in hospitals, metro stations, universities, and disaster relief areas. It was created to address limitations of the earlier Aura-Kiosk v1.0 system, where hardware upgrades, payment integrations, and new features required major code changes, making maintenance difficult. Aura Retail OS aims to build a flexible, modular kiosk platform that adapts easily to hardware changes, business policies, and emergencies without disrupting the entire system. Following the Path B — Modular Hardware Platform approach, it emphasizes scalability, low coupling, maintainability, and extensibility using Object-Oriented Design principles and patterns. The system includes separate subsystems such as Kiosk Core, Inventory, Payment, Hardware Abstraction Layer, and City Monitoring. These communicate through interfaces and event-driven methods to ensure organization and scalability. Core services like CentralRegistry, EventBus, and PersistenceManager support system-wide functions and configurations.

Aura Retail OS accommodates various kiosk types like Food, Pharmacy, and Emergency Relief Kiosks, each with its own rules, pricing, and inventory management. It offers features like dynamic hardware replacement, secure inventory handling, multiple payment options, and nested inventory bundles, employing design patterns such as Bridge, Decorator, Adapter, Composite, and Proxy. The React-based frontend dashboard connects to a Java backend via REST APIs, allowing users to monitor kiosk operations, hardware status, inventory, transactions, payment providers, and system events. The project exemplifies key object-oriented programming principles—Encapsulation, Abstraction, Inheritance, and Polymorphism—while solving practical system design challenges.

1.2 Business Requirements :

- Functional Requirements :
    - The system must support multiple kiosk types such as Food Kiosks, Pharmacy Kiosks, and Emergency Relief Kiosks.
    - The platform must allow hardware components to be replaced or upgraded without changing the main system logic.

- The inventory system must support both individual products and nested product bundles.
- The system must integrate multiple payment methods including Credit Card, UPI, Wallet, and Crypto payments.
- The kiosk interface must provide simple operations such as:
  - Purchase Item
  - Refund Transaction
  - Run Diagnostics
  - Restock Inventory
- The system must support optional hardware modules like refrigeration, solar monitoring, and network connectivity.
- All inventory access must pass through a secure validation layer with logging and authorization checks.
- The system must support event-based communication between monitoring services and kiosks.
- The platform must support dynamic pricing policies such as standard, discounted, and emergency pricing.
- The system must store inventory data, transaction history, and configuration details using persistent storage.

- Non-functional Requirements:
  - The system must keep a low level of coupling across all subsystems.
  - The architecture should be modular and easily extendable for future upgrades.
  - The platform needs to support large-scale deployment across multiple city locations.
  - The system must guarantee thread safety for shared services and global resources.
  - Transactions should be atomic to prevent inconsistent inventory or payment states.
  - The platform should include fault recovery and failure handling mechanisms.
  - The frontend dashboard must offer real-time monitoring and system updates.
  - New payment providers should integrate seamlessly without changes to existing payment logic.
  - Product availability should be dynamically calculated based on stock, reservations, and hardware status.
  - The project should demonstrate clear adherence to Object-Oriented Design principles and patterns. patterns.

1.3 Technical Challenges Faced :

- **Maintaining Low Coupling Between Subsystems:** A key challenge was to design separate subsystems for hardware, inventory, payment, and monitoring without forming tight couplings. The team addressed this by extensively utilizing interfaces, event-driven communication, and design patterns like Facade, Observer, and Bridge to ensure modules remained loosely connected.

- **Integrating Multiple Incompatible Payment APIs :** Various payment providers like Credit Card, UPI, Wallet, and Crypto systems used entirely different APIs and communication methods. Developing a unified payment interface without altering the existing kiosk logic was challenging, necessitating the implementation of an Adapter-based architecture for smooth integration.

- **Supporting Dynamic Hardware Replacement :** The system was designed to allow runtime replacement of dispenser hardware such as spiral dispensers, robotic arms, and conveyor dispensers without disrupting kiosk operations. To ensure this flexibility within a clean architecture, careful implementation of the Bridge and Decorator patterns was necessary.

- **Ensuring Inventory Consistency During Transactions :** Handling transaction failures without corrupting inventory data was a major challenge. The system needed to ensure atomic transactions so that payment, dispensing, and stock updates either all complete or are safely rolled back, employing Command and Memento-based recovery mechanisms.

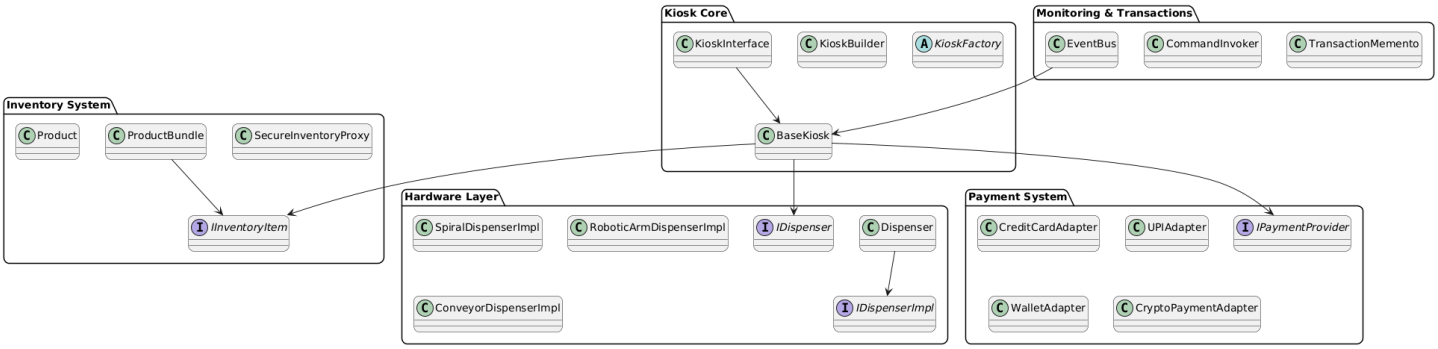
- **Managing Nested Product Bundles Efficiently :** The inventory system needed to handle complex product bundles, where one bundle could include other bundles and products. Ensuring accurate stock levels and price calculations across multiple levels of nesting required implementing recursive inventory management with the Composite design pattern.

2. Sub - System Architecture:

Aura Retail OS employs a modular layered architecture, dividing the system into independent subsystems to enhance scalability, flexibility, and reduce coupling. Each subsystem manages a specific aspect of the kiosk ecosystem and interacts with others via interfaces and event-driven communication. This approach improves maintainability and facilitates the integration of new features or hardware without disrupting existing business logic.

The system architecture comprises five primary subsystems: Kiosk Core, Hardware Abstraction Layer, Inventory, Payment, and Monitoring & Transaction Management Systems. A centralized Foundation Layer offers services like CentralRegistry, EventBus, and PersistenceManager to coordinate global configuration, event exchange, and storage across the platform. The Kiosk Core manages kiosk creation, operations, and user interactions. The Hardware Layer hides low-level hardware details from the business logic. The Inventory System tracks products, bundles, and stock levels. The Payment System connects with various external payment providers through a unified interface. Meanwhile, the Monitoring System oversees event logging, failure detection, and transaction monitoring across the city-wide deployment.

High level System Architecture:



2.1 Kiosk Core Sub-system :

The Kiosk Core subsystem serves as the central operational component of Aura Retail OS. It manages kiosk behavior, user interaction, purchase processing, and coordination among subsystems. The design centers on the BaseKiosk abstract class, which defines the common workflow for all kiosk types while supporting specialized behavior through inheritance and method overriding.

Three specialized kiosk types are implemented:

- FoodKiosk
- PharmacyKiosk
- EmergencyReliefKiosk

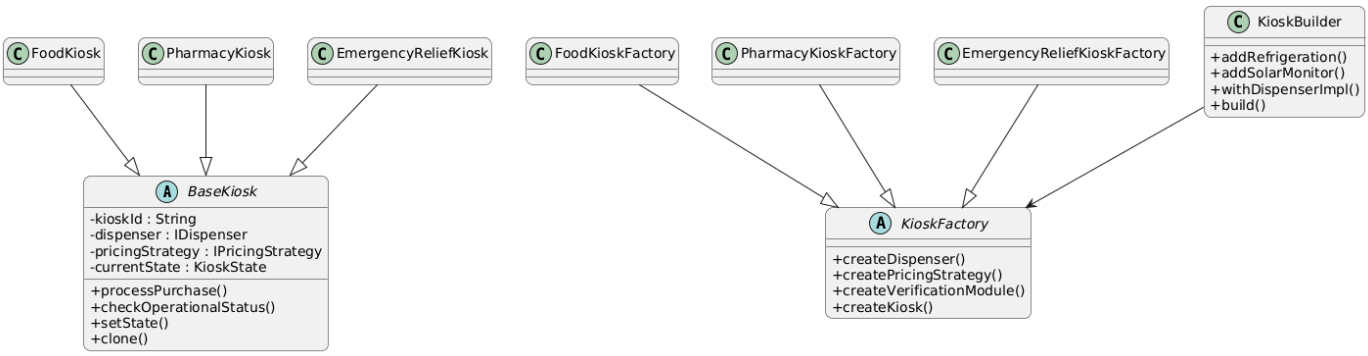
Each kiosk inherits from BaseKiosk and enforces its specific policies, verification processes, and operational limits. For instance, PharmacyKiosk verifies prescriptions, while EmergencyReliefKiosk imposes limits on emergency purchases.

The subsystem employs the Abstract Factory pattern via KioskFactory and its specific implementations to produce compatible kiosk components such as dispensers, pricing strategies, and verification modules. Additionally, the Builder pattern is used through KioskBuilder, which allows optional hardware modules like refrigeration and solar monitoring to be added dynamically during kiosk assembly.

The KioskInterface class acts as a Facade layer that exposes simplified operations such as:

- purchaseItem()
- refundTransaction()
- runDiagnostics()
- restockInventory()

This design reduces complexity for external systems and maintains low coupling between subsystems as it can be seen in the diagram given below :



2.2 Hardware Abstraction Layer:

The Hardware Abstraction Layer (HAL) isolates hardware-specific details from the main kiosk business logic. This subsystem allows hardware devices to be replaced, upgraded, or extended without altering the core operational code. The Bridge pattern is realized through the Dispenser abstraction and various IDispenserImpl implementations, including:

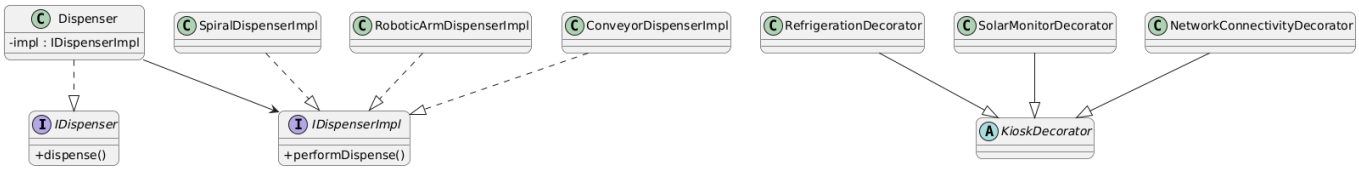
- SpiralDispenserImpl
- RoboticArmDispenserImpl
- ConveyorDispenserImpl

This structure allows dispenser hardware to be swapped dynamically during runtime while keeping kiosk logic unchanged.

Hardware modules are integrated using the Decorator pattern, which includes classes like:

- RefrigerationDecorator
- SolarMonitorDecorator
- NetworkConnectivityDecorator

These classes extend kiosk capabilities dynamically without altering the original kiosk classes. This method prevents subclass explosion and allows for modular hardware expansion. Additionally, the subsystem employs Proxy-based hardware validation to ensure that products requiring specific hardware modules are disabled if the necessary hardware fails. The below given diagram helps in understanding the architecture:



2.3 Inventory and Payment Sub-system :

The Inventory System oversees product storage, stock monitoring, bundle management, and security across all kiosks. It supports both single products and nested product bundles through the Composite pattern.

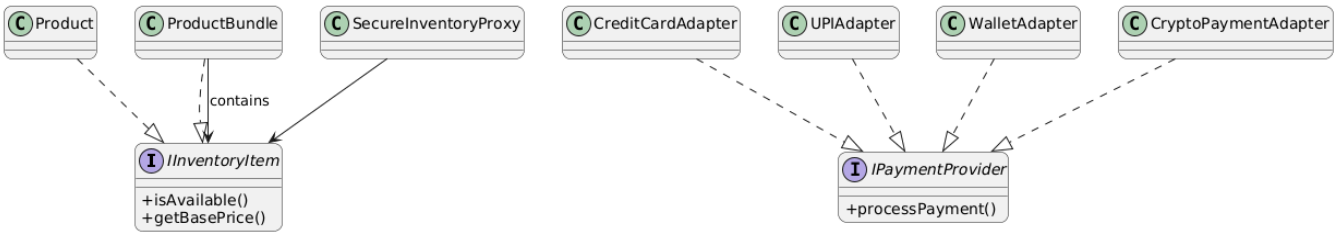
The Product class models individual inventory items, while ProductBundle functions as a composite container that can hold both products and other bundles. This design enables the system to represent complex inventory setups, like emergency kits that include medical supplies and various other items. Stock levels and pricing are calculated in real-time, considering inventory counts, reserved stock, ongoing transactions, and hardware availability. Access to inventory is protected by SecureInventoryProxy, which manages authorization, logs access activities, and blocks unauthorized modifications.

The Payment System connects various third-party payment providers via an Adapter-based architecture. APIs like:

- Credit Card Gateway
- UPI System
- Digital Wallet
- Crypto API

are standardized into a single IPaymentProvider interface, enabling new payment options to be integrated without changing the kiosk logic.

Pricing flexibility is enabled through the Strategy pattern, which includes StandardPricingStrategy, DiscountedPricingStrategy, and EmergencyPricingStrategy, allowing pricing rules to adapt dynamically to system conditions.



2.4 Monitoring and Transaction management Sub - system:

The Monitoring & Transaction Management subsystem manages event communication, transaction processing, rollback control, and system monitoring. It is built with an event-driven architecture to minimize direct module dependencies. The EventBus class functions as the main event dispatcher within the Observer pattern, handling events such as:

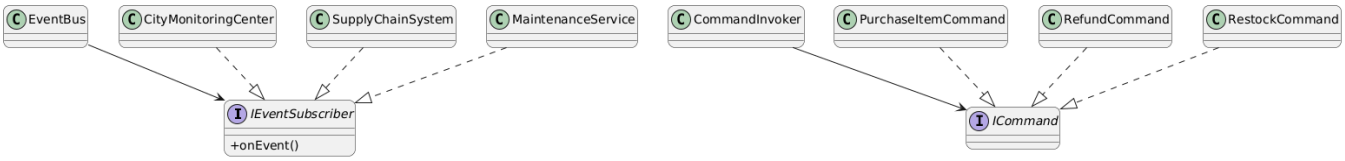
- HardwareFailureEvent
- LowStockEvent
- TransactionCompletedEvent
- EmergencyModeActivatedEvent

are published to subscribed monitoring services including:

- CityMonitoringCenter
- SupplyChainSystem
- MaintenanceService

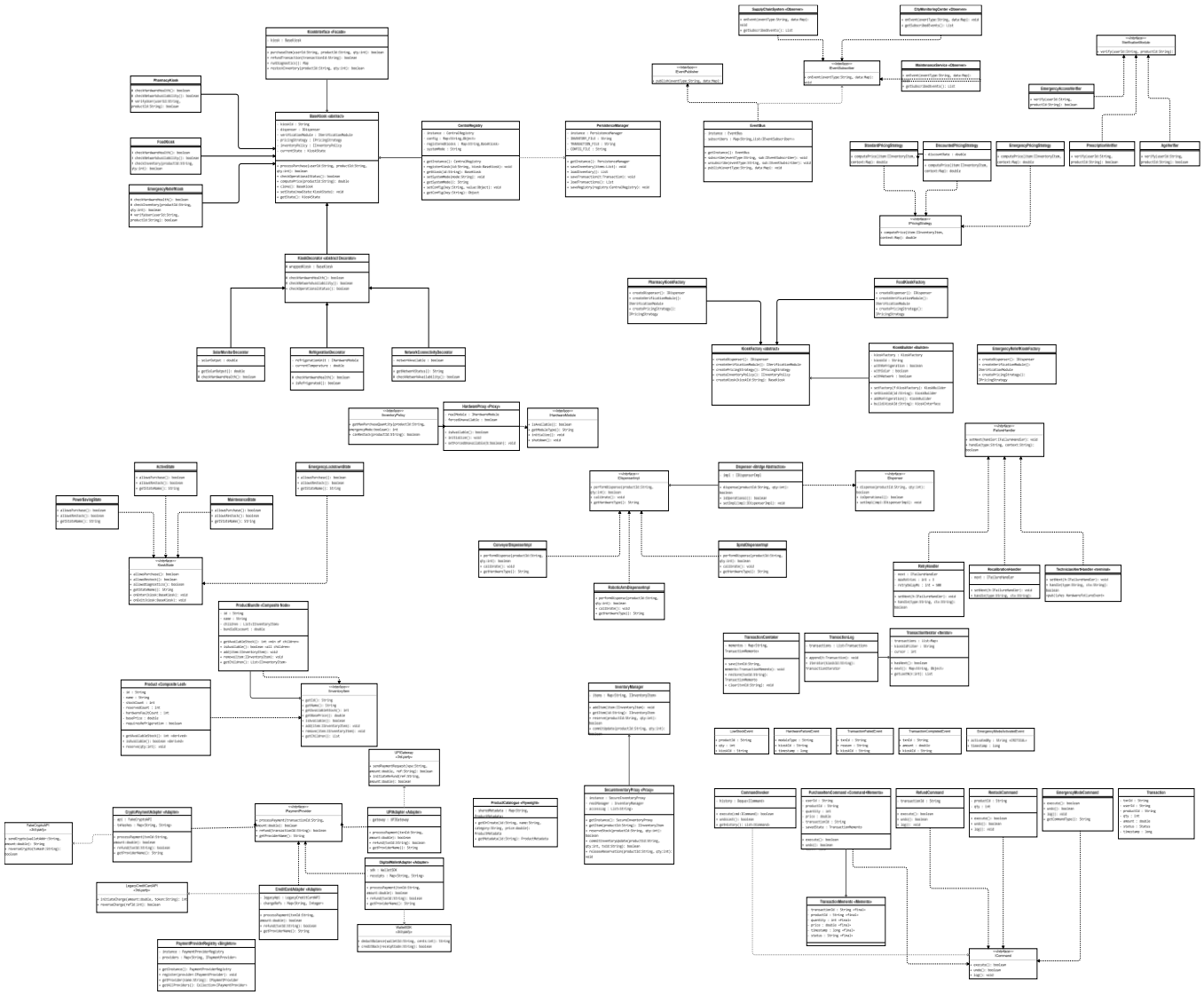
Transaction execution employs the Command pattern, encapsulating operations like purchases, refunds, and restocking as command objects such as PurchaseItemCommand, RefundCommand, and RestockCommand. The CommandInvoker handles execution history and allows undo actions. To ensure transaction consistency, the Memento pattern is used via TransactionMemento and TransactionCaretaker, saving the system's state before each transaction so inventory and transaction data can be restored if needed.

The subsystem also uses Iterator-based transaction traversal and the Chain of Responsibility for hardware failure recovery.

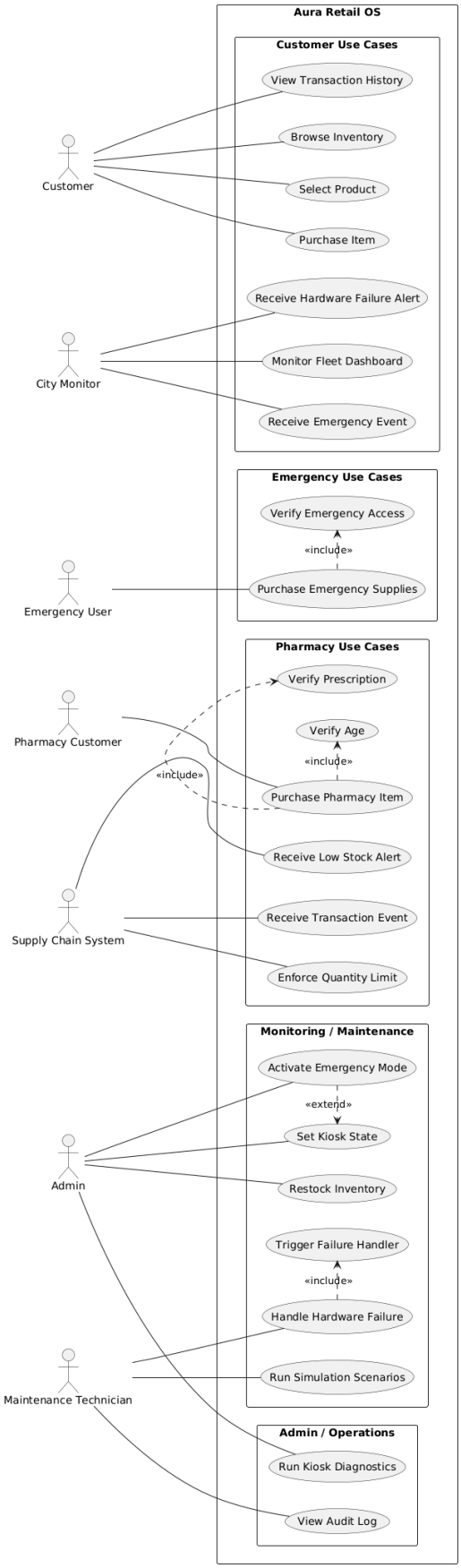


3. UML Diagrams: (Note : Please click on the diagrams to get redirected to the link of the diagrams)

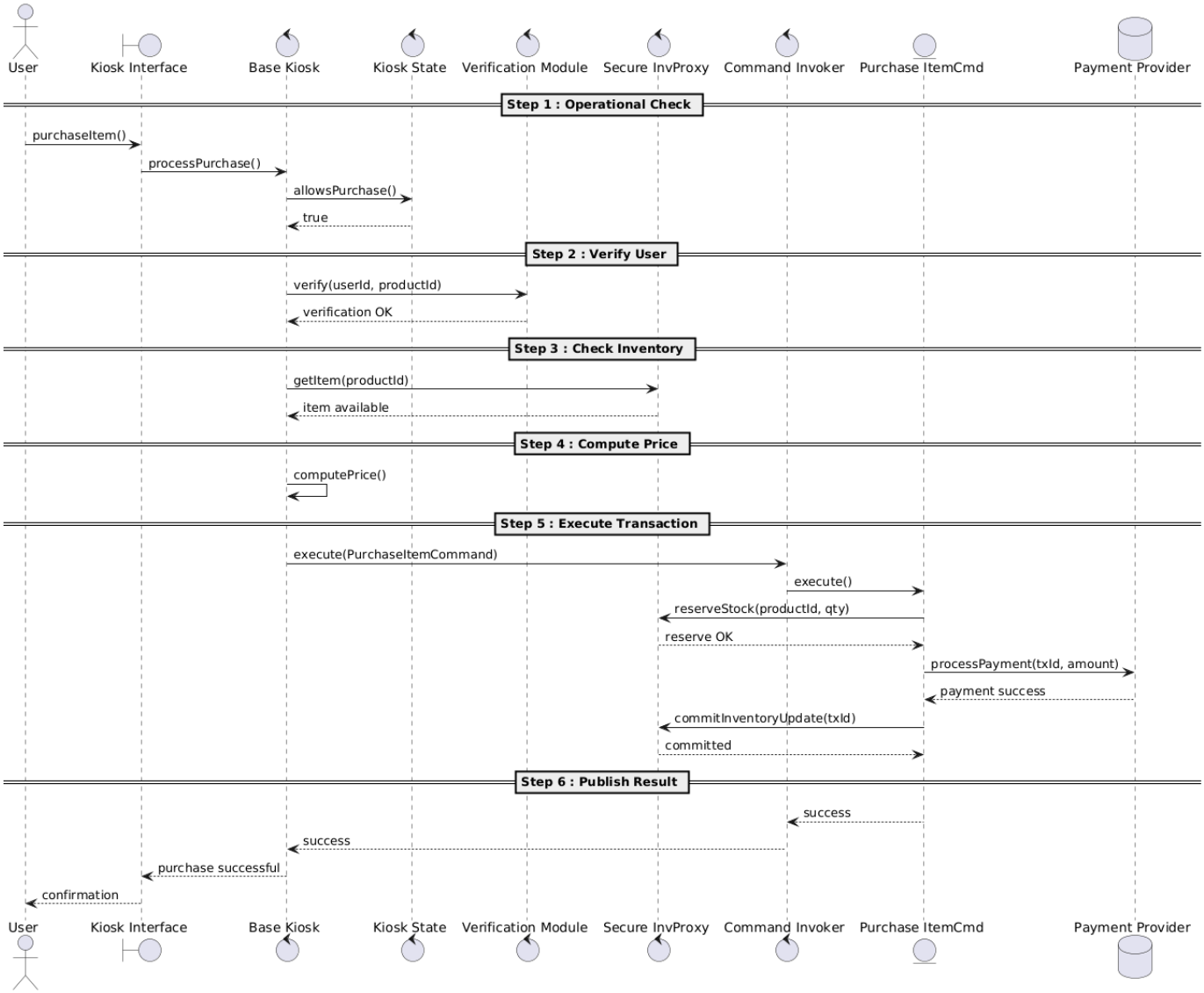
3.1 Class Diagram :



3.2 Use - Case Diagram :



3.3 Sequence Diagram :



4. Design Pattern implementation justification :

4.1 Singleton design pattern:

Classes include: CentralRegistry, EventBus, PersistenceManager, SecureInventoryProxy, and PaymentProviderRegistry. Using the Singleton pattern makes sure there is only one instance of each class within the JVM, ensuring a consistent shared state across the system.

Implementation :

```
// CentralRegistry.java — double-checked locking with volatile

private static volatile CentralRegistry instance;



public static CentralRegistry getInstance() {

   if (instance == null) {

      synchronized (CentralRegistry.class) {

         if (instance == null) instance = new CentralRegistry();

      }

   }

   return instance;

}

// private final Map<String, BaseKiosk> registeredKiosks

// private String systemMode
```

The 'volatile' keyword stops instruction reordering, making sure that the partially-built object is never accessible to other threads. This approach is used consistently across all five classes.

Without Singleton :

Using multiple CentralRegistry instances could lead to inconsistent kiosk maps and system modes. When setSystemMode("EMERGENCY") is called on one instance, it wouldn't update other parts of the system, risking unnoticed safety issues during a city-wide emergency.

4.2 Abstract Factory design pattern:

Classes include KioskFactory (abstract) and its subclasses FoodKioskFactory, PharmacyKioskFactory, and EmergencyReliefKioskFactory. The Abstract Factory pattern enables the creation of related object families without defining specific classes.

Implementation

```
// KioskFactory.java (abstract)

public abstract IDispenser         createDispenser();

public abstract IVerificationModule createVerificationModule();

public abstract IPricingStrategy    createPricingStrategy();

public abstract IInventoryPolicy    createInventoryPolicy();

public BaseKiosk createKiosk(String kioskId) { ... } // calls Factory Method
```

PharmacyKioskFactory produces PrescriptionVerifier along with StandardPricingStrategy, while EmergencyReliefKioskFactory offers EmergencyAccessVerifier and EmergencyPricingStrategy. The type system enforces architectural separation, preventing mixing components from different factories.

Without Abstract Factory

KioskBuilder depends on if-else chains to choose compatible components. Introducing a new kiosk type requires changing the builder, which directly breaches the Open/Closed Principle.

4.3 Factory Method design pattern:

The Factory Method pattern establishes an interface for creating objects while allowing subclasses to determine the specific class to instantiate. Within KioskFactory.createKiosk(), the method call instantiateKiosk(kioskId) serves as the Factory Method — each subclass overrides this method to return either a FoodKiosk, PharmacyKiosk, or EmergencyReliefKiosk.

Without Factory Method

The parent factory should include a type parameter or switch statement to select the kiosk class, linking the abstract base factory to all three concrete kiosk classes.

4.4 Builder design pattern:

The Builder pattern decouples the process of constructing complex objects from their representation and offers a fluent API to assemble a fully configured kiosk.

Implementation :

```
// KioskBuilder.java — fluent method chaining

public KioskBuilder addRefrigeration()            { withRefrigeration = true; return this; }

public KioskBuilder addSolarMonitor()             { withSolar = true;        return this; }

public KioskBuilder addNetworkModule()            { withNetwork = true;      return this; }

public KioskBuilder withDispenserImpl(IDispenserImpl impl) { ... return this; }

public KioskInterface build(String kioskId) { ... }  // applies decorators in order


// Simulation usage:

KioskInterface kiosk = new KioskBuilder(new FoodKioskFactory())

    .addRefrigeration()

    .addSolarMonitor()

    .withDispenserImpl(new RoboticArmDispenserImpl())

    .build("F-003");
```

The build() call applies decorators sequentially: Refrigeration, then Solar, then Network, resulting in NetworkConnectivityDecorator(SolarMonitorDecorator(RefrigerationDecorator(FoodKiosk))).

Without Builder

A constructor that includes four boolean flags along with an optional dispenser implementation would be necessary — this is an example of the telescoping constructor anti-pattern, which leads to unreadable call sites as more options are incorporated.

4.5 Prototype design pattern:

Implementation :

```
// BaseKiosk.java implements Cloneable

@Override

public BaseKiosk clone() {

   try { return (BaseKiosk) super.clone(); }

   catch (CloneNotSupportedException e) { throw new RuntimeException(e); }

}
```

A pre-configured kiosk, with all components already set, can be quickly duplicated for deployment at a new site without needing to go through the full factory and builder process.

Without Prototype

Deploying identical kiosks at 50 metro stations involves 50 factory calls and 50 builder chains, increasing the risk of configuration drift.

4.6 Adapter design pattern:

Classes include: CreditCardAdapter, DigitalWalletAdapter, UPIAdapter, and CryptoPaymentAdapter. The Adapter pattern transforms the interface of a class into another that clients anticipate. Four incompatible third-party APIs are each encapsulated by an Adapter that implements the consistent IPaymentProvider interface.

Implementation :

```
// CreditCardAdapter.java

public class CreditCardAdapter implements IPaymentProvider {
```

```
    private final LegacyCreditCardAPI legacyApi;


    @Override
    public boolean processPayment(String transactionId, double amount) {
        int ref = legacyApi.initiateCharge(amount, transactionId); // incompatible signature
        if (ref != -1) { chargeRefs.put(transactionId, ref); return true; }
        return false;
    }
    @Override public boolean refund(String transactionId) { ... }
}
```

Critical proof: CryptoPaymentAdapter connects FakeCryptoAPI.sendCrypto(String walletAddress, double amount) to IPaymentProvider.processPayment(String txId, double amount), despite their different signatures. This adapter was registered at runtime through PaymentProviderRegistry.register(cryptoAdapter), without modifying any existing files.

Without Adapter

KioskInterface requires unique method signatures for each payment provider. Introducing a new payment method entails modifying BaseKiosk, KioskInterface, and all command classes, thus breaking the Open/Closed Principle.

4.7 Bridge design pattern:

Classes include Dispenser as an abstraction, with implementations like SpiralDispenserImpl, RoboticArmDispenserImpl, and ConveyorDispenserImpl. The Bridge pattern separates the abstraction from its implementation, allowing them to change independently.

Implementation :

```
// Dispenser.java (abstraction)
public class Dispenser implements IDispenser {
```

```
    private IDispenserImpl impl; // Bridge: holds reference to implementation

    public void setImpl(IDispenserImpl impl) { this.impl = impl; }

    @Override

    public boolean dispense(String productId, int qty) {

        return impl.performDispense(productId, qty); // delegates entirely

    }

}
```

In KioskBuilder.build(), .withDispenserImpl(new RoboticArmDispenserImpl()) replaces the hardware implementation without any modifications to BaseKiosk, KioskInterface, or any factory. Simulation output: [BRIDGE] Dispenser swap required zero changes to BaseKiosk or KioskInterface.

Without Bridge

Including RoboticArmDispenserImpl involves subclassing Dispenser into RoboticArmDispenser and then creating new kiosk subclasses — resulting in a class explosion of at least O(kiosk_types x dispenser_types): a minimum of 9 classes for 3 kiosk types and 3 dispenser implementations.

4.8 Composite design pattern:

Classes include Product (Leaf) and ProductBundle (Composite), both implementing IInventoryItem. They support N-level nesting through a consistent interface.

Implementation :

```
// ProductBundle.java — recursive operations on children

@Override public boolean isAvailable() {

    return !children.isEmpty()

        && children.stream().allMatch(IInventoryItem::isAvailable);

}

@Override public int getAvailableStock() {
```

```
    return children.stream().mapToInt(IInventoryItem::getAvailableStock).min().orElse(0);
}
@Override public double getBasePrice() {
    double total = children.stream().mapToDouble(IInventoryItem::getBasePrice).sum();
    return total * (1.0 - bundleDiscount);
}
```

The Scenario C inventory tree: EmergencyKit -> MedKit -> [Bandages, Antiseptic, Paracetamol]. When Paracetamol stock depletes to zero, paracetamol.isAvailable() becomes false, which then propagates: medKit.isAvailable() becomes false, followed by emergencyKit.isAvailable() also becoming false — all via the same recursive interface call.

Without Composite

The system must handle single products and bundles separately. Each inventory query would need type-checking with instanceof and manual recursion outside the objects.

4.9 Decorator design pattern:

Classes include: KioskDecorator (abstract), RefrigerationDecorator, SolarMonitorDecorator, and NetworkConnectivityDecorator. The Decorator pattern allows for adding responsibilities to an object dynamically.

Implementation :

```
// KioskDecorator.java (abstract) — extends AND wraps BaseKiosk
public abstract class KioskDecorator extends BaseKiosk {
    protected final BaseKiosk wrappedKiosk;
    public KioskDecorator(BaseKiosk kiosk) {
        super(kiosk.getKioskId());
        this.wrappedKiosk = kiosk;
        this.setDispenser(kiosk.getDispenser()); // inherits all components
```

```
    }
}

// RefrigerationDecorator.java
@Override protected boolean checkHardwareHealth() {
    if (!refrigerationUnit.isAvailable()) {
        System.out.println("[RefrigerationDecorator] Refrigeration failed — blocking purchases.");
        return false;
    }
    return wrappedKiosk.checkOperationalStatus();
}
```

Critical design proof: FoodKiosk.java was never altered to include refrigeration support. The decorator handles the hardware check entirely externally. Simulation confirms: [DECORATOR] FoodKiosk.java remains unmodified..

Without Decorator

Three hardware modules across three kiosk types would need at least nine subclasses, increasing O(modules × kiosk_types) with each additional hardware component.

4.10 Façade design pattern:

The Facade offers a streamlined interface with four methods to access a subsystem that includes over 15 internal classes. KioskInterface only reveals purchaseItem(), refundTransaction(), runDiagnostics(), and restockInventory().

Implementation :

```
// KioskInterface.java
// PROOF: grep -rn 'import aura.hardware' KioskInterface.java -> 0 results
//        grep -rn 'import aura.payment'  KioskInterface.java -> 0 results
```

```
public class KioskInterface {

    private final BaseKiosk kiosk;


    public boolean purchaseItem(String userId, String productId, int quantity) { ... }

    public boolean refundTransaction(String transactionId) { ... }

    public Map<String, Object> runDiagnostics() { ... }

    public boolean restockInventory(String productId, int quantity) { ... }

}
```

Without Facade

External callers (such as REST endpoints, simulation classes, or the React dashboard) would need direct knowledge of CommandInvoker, TransactionIterator, PersistenceManager, and over 12 other classes to execute a single purchase.

4.11 Proxy design pattern:

Implementation :

```
// SecureInventoryProxy.java

public boolean reserveStock(String productId, int qty) {

    logAccess("RESERVE", productId + " qty=" + qty); // mandatory audit log

    return realManager.reserve(productId, qty);

}

public boolean commitInventoryUpdate(String productId, int qty, String transactionId) {

    if (transactionId == null || transactionId.isEmpty()) return false; // Constraint 4

    return realManager.commitUpdate(productId, qty, transactionId);

}
```

SecureInventoryProxy enforces three main concerns: (a) audit logging for every access, (b) validation of transactionId prior to commit, and (c) hardware dependency verification through Product.isAvailable(), which checks its HardwareProxy reference. HardwareProxy activates the Chain of Responsibility in case of a module failure.

Without Proxy

Inventory could decrease even without a valid transaction ID, requiring audit logging to be duplicated across all classes handling inventory.

4.12 Flyweight design pattern:

ProductCatalogue serves as a shared collection for product metadata, including intrinsic state such as name, base price, and category. Each kiosk maintains its own instance-specific stock count, representing extrinsic state, stored separately. Without implementing Flyweight, deploying 50 kiosks across a city with 200 product types would result in 10,000 redundant metadata objects.

Without Flyweight

Memory usage will be O(kiosks x products) for metadata, compared to O(products) with Flyweight. The garbage collection pressure on the long-running JVM kiosk process would increase proportionally.

4.13 Command design pattern:

Classes include CommandInvoker, PurchaseItemCommand, RefundCommand, RestockCommand, and EmergencyModeCommand. Each operation is encapsulated as an object, allowing for undo functionality and thread-safe execution.

Implementation :

```
// CommandInvoker.java

public synchronized boolean execute(ICommand command) {

    boolean success = command.execute();

    command.log();
```

```
    if (success) history.push(command);
    return success;
}
public boolean undoLast() {
    if (history.isEmpty()) return false;
    return history.pop().undo();
}
```

CommandInvoker is designed to be thread-safe. The method PurchaseItemCommand.undo() reverses both inventory reservation and payment by utilizing a TransactionMemento. Additionally, EmergencyModeCommand alters the system state and can be reversed.

Without Command

Implementing undo would necessitate duplicating rollback logic across all operation classes and replicating the synchronized history stack in every instance.

4.14 Observer design pattern:

Classes: EventBus (Subject), CityMonitoringCenter, SupplyChainSystem, MaintenanceService (Observers). This setup separates event producers from consumers, eliminating direct class dependencies.

Implementation :

```
// EventBus.java
public void publish(String eventType, Map<String, Object> eventData) {
    List<IEventSubscriber> subs = subscribers.getOrDefault(eventType, Collections.emptyList());
    for (IEventSubscriber sub : new ArrayList<>(subs)) {
        sub.onEvent(eventType, eventData);
    }
}
```

```
// Wiring subscriptions in SystemBootstrap.java:

bus.subscribe("HardwareFailureEvent", monitor);

bus.subscribe("HardwareFailureEvent", maintenance);

bus.subscribe("LowStockEvent", supplyChain);

bus.subscribe("EmergencyModeActivatedEvent", (type, data) -> {

   CentralRegistry.getInstance().getAllKiosks().values()

      .forEach(k -> k.setState(new EmergencyLockdownState()));

});
```

There are five event types: HardwareFailureEvent, LowStockEvent, TransactionCompletedEvent, TransactionFailedEvent, EmergencyModeActivatedEvent. The emergency event is sent synchronously with high priority — all kiosks switch to EmergencyLockdownState before any other processing.

Without Observer

Each class that modifies state would require direct references to monitoring systems, resulting in over 15 tight dependencies. This design makes testing and extending the system independently unfeasible.

4.15 Strategy design pattern:

Implementation :

```
// IPricingStrategy.java

public interface IPricingStrategy {

   double computePrice(IInventoryItem item, Map<String, Object> context);

}

// EmergencyPricingStrategy checks:

//   context.get("systemMode").equals("EMERGENCY") -> apply price caps
```

Strategies are generated by the factory and can be changed at runtime through BaseKiosk.setPricingStrategy().

Without Strategy

The pricing logic in BaseKiosk.computePrice() relies on nested if-else chains, so any updates to pricing rules necessitate modifying this key class in the codebase.

4.16 State design pattern:

Classes include: KioskState (interface), ActiveState, MaintenanceState, PowerSavingState, and EmergencyLockdownState. Each class defines its specific permission rules.

Implementation :

```
// KioskState.java (interface)

public interface KioskState {

    boolean allowsPurchase();

    boolean allowsRestock();

    boolean allowsDiagnostics();

    String getStateName();

    void onEnter(BaseKiosk kiosk);

    void onExit(BaseKiosk kiosk);

}
```

State	allowsPurchase	allowsRestock	allowsDiagnostics	Trigger
ActiveState	Yes	Yes	Yes	Kiosk created
MaintenanceState	No	Yes	Yes	Maintenance command
PowerSavingState	Yes	No	Yes	Inactivity timeout

EmergencyLockdownState	No	No	Yes	EmergencyModeActivatedEvent

Without State

BaseKiosk.processPurchase() involves 3 permission types and 4 states, leading to 12 nested conditionals, which becomes unmanageable as new states are introduced.

4,17 Template method design pattern:

The method is final, meaning subclasses cannot change the order of steps. It outlines the core algorithm structure, while subclasses customize specific points through hooks.

Implementation :

```
// BaseKiosk.java

public final boolean processPurchase(String userId, String productId, int quantity) {

   if (!checkOperationalStatus()) return false; // Step 1: hardware, state, network checks

   if (!verifyUser(userId, productId)) return false; // Step 2: HOOK — specific to kiosk type

   if (!checkInventory(productId, quantity)) return false; // Step 3: HOOK

   double price = computePrice(productId); // Step 4: Strategy

   return executeTransaction(userId, productId, quantity, price); // Step 5: Command

}

protected boolean verifyUser(String userId, String productId) { ... } // can be overridden

protected abstract boolean checkHardwareHealth();

protected abstract boolean checkNetworkAvailability();
```

PharmacyKiosk.verifyUser() calls super.verifyUser() then adds PrescriptionVerifier. EmergencyReliefKiosk.checkInventory() enforces quantity caps via EmergencyInventoryPolicy.

Without Template Method

Each of the three kiosk types will have its own purchase flow. The security check skipping — the same bug from v1 — would reappear due to code divergence.

4.18 Iterator design pattern:

```
// TransactionIterator.java

public class TransactionIterator {

   private final List<Transaction> filtered;

   public TransactionIterator(List<Transaction> all, String kioskId) {

      this.filtered = all.stream()

         .filter(t -> t.getKioskId().equals(kioskId))

         .collect(toList());

   }

   public List<Transaction> getLastN(int n) { ... }

}
```

This is used in KioskInterface.runDiagnostics() and the GET /api/kiosks/{id} REST endpoint. It iterates over kiosk-specific transactions without exposing the internals of TransactionLog.

Without Iterator

Using runDiagnostics() requires internal knowledge of the TransactionLog data structure, which breaches encapsulation and the Single Responsibility Principle.

4.19 Chain of Responsibility design pattern:

Implementation :

```
// RetryHandler.java

public boolean handle(String failureType, String context) {
```

```
    if ("TRANSIENT".equals(failureType)) {
        System.out.println("[RetryHandler] Retry succeeded.");
        return true;
    }
    System.out.println("[RetryHandler] Cannot handle. Passing to next handler.");
    if (next != null) return next.handle(failureType, context);
    return false;
}
```

There are three handlers: RetryHandler manages transient failures, RecalibrationHandler addresses sensor drift, and TechnicianAlertHandler escalates issues to a human technician. HardwareProxy initiates this chain when a module health check fails.

Without Chain of Responsibility

All failure types are handled within a single class using a switch statement. To add a new failure type, you must modify the existing handler class, which violates the Open/Closed Principle.

5. System Constraints - Enforced Mechanically :

Four hard constraints are enforced through the call chain — not as documentation or API-boundary validation. Each trace shows the exact point where the constraint produces a false return.

5.1 Constraint - 1 : Emergency Purchase quantity cap

Rule: During emergency mode, users cannot buy more than 2 units of any item.

Decision Flow :

```
purchaseItem("user1", "WATER-001", 5)
  -> BaseKiosk.processPurchase()
    -> BaseKiosk.checkInventory("WATER-001", 5)
      -> EmergencyReliefKiosk.checkInventory() calls EmergencyInventoryPolicy
        -> EmergencyInventoryPolicy.getMaxPurchaseQuantity("WATER-001", true) -> 2
        -> 5 > 2 -> return false
      -> checkInventory() returns false
    -> processPurchase() returns false
    -> PurchaseItemCommand is never created intiated
```

5.2 Constraint - 2 : Pharmacy Products Require Prescription

Rule: A valid prescription must be on record to purchase from a pharmacy kiosk.

Decision Flow :

```
purchaseItem("user1", "MED-001", 1)
  -> BaseKiosk.processPurchase()
    -> PharmacyKiosk.verifyUser("user1", "MED-001")
      -> super.verifyUser()
        -> verificationModule.verify("user1", "MED-001")
```

-> PrescriptionVerifier.verify()

-> checks prescription database

-> no prescription found -> return false

-> verifyUser() returns false

-> processPurchase() returns falseturns false

5.3 Constraint - 3 : Refrigeration Hardware Dependency

Rule: Refrigerated items cannot be bought if the refrigeration unit is not functioning properly.

Decision Flow :

purchaseItem("user1", "DAIRY-001", 1)

-> BaseKiosk.processPurchase()

-> RefrigerationDecorator.checkHardwareHealth()

-> refrigerationUnit.isAvailable()

-> HardwareProxy.isAvailable()

-> RefrigerationUnit.isAvailable() -> false (hardware failed)

-> return false

-> RefrigerationDecorator returns false, blocking the purchase

-> processPurchase() returns false

Note: Product.isAvailable() also checks refrigerationProxy independently.

5.4 Constraint - 4 : Inventory Commit requires valid Transaction ID

Rule: Stock can only be reduced permanently once a valid transaction ID is generated and payment has been confirmed.

Decision Flow :

```
PurchaseItemCommand.execute()

  -> Step 1: TransactionMemento created (immutable snapshot)

  -> Step 2: SecureInventoryProxy.reserveStock("FOOD-001", 2) -> soft-reserve

  -> Step 3: PaymentProviderRegistry.getProvider().processPayment(txId, amount)

     -> if payment fails: undo() restores from memento, returns false

  -> Step 4: SecureInventoryProxy.commitInventoryUpdate("FOOD-001", 2, txId)

     -> if (txId == null || txId.isEmpty()) -> return false  // CONSTRAINT ENFORCEMENT

     -> else -> InventoryManager.commitUpdate() -> stock permanently decremented
```

6. Simulation Scenarios walkthrough :

Build and Run Instruction :

```
# Build the project

mvn clean compile

# Scenario A — Decorator + Bridge

mvn exec:java –“Dexec.mainClass=aura.simulation.ScenarioA_NewHardwareModule"

# Scenario B — Adapter (runtime registration)

mvn exec:java –“Dexec.mainClass=aura.simulation.ScenarioB_NewPaymentProvider"

# Scenario C — Composite (nested bundle propagation)

mvn exec:java –“Dexec.mainClass=aura.simulation.ScenarioC_NestedBundles"

# Full Integration Demo (all patterns)

mvn exec:java –“Dexec.mainClass=aura.simulation.SubTask2Demo"

# Foundation Layer Test

mvn exec:java –“Dexec.mainClass=aura.simulation.FoundationTest"
```

6.1 Scenario - A : Adding a new Hardware Module

- Patterns demonstrated: Decorator, Bridge, Abstract Factory, Facade
- KioskBuilder(new FoodKioskFactory()).build("KIOSK-001") creates a basic food kiosk with SpiralDispenser and no modules.
- KioskInterface.runDiagnostics() verifies the initial setup.
- Bridge swap: .withDispenserImpl(new RoboticArmDispenserImpl()) — no changes to BaseKiosk or any factory.
- Decorator stacking: .addRefrigeration().addSolarMonitor() results in SolarMonitorDecorator(RefrigerationDecorator(FoodKiosk)). FoodKiosk.java remained unchanged.

Expected Console Output :

[SystemBootstrap] All observers registered.

[State] KIOSK-001 -> ACTIVE

[FACTORY] FoodKiosk KIOSK-001 created without refrigeration.

[FACADE] Diagnostics: {kioskId=KIOSK-001, operationalStatus=true, ...}

[BRIDGE] Swapping SpiralDispenser -> RoboticArmDispenser on KIOSK-001...

[RefrigerationDecorator] Module attached to KIOSK-002

[DECORATOR] FoodKiosk KIOSK-002 created with Refrigeration + Solar modules.

[BRIDGE] KIOSK-002 dispenser is now RoboticArm.

[DECORATOR] FoodKiosk.java was NOT modified. Module added via Decorator.

[BRIDGE] Dispenser swap required zero changes to BaseKiosk or KioskInterface.

6.2 Scenario - B : Integrating a new Payment Method

- Patterns demonstrated: Adapter, Strategy, Command, Facade
- PaymentProviderRegistry lists existing providers: CreditCard, UPI, Wallet.
- New CryptoPaymentAdapter(new FakeCryptoAPI()) wraps FakeCryptoAPI.sendCrypto(walletAddress, amount) behind IPaymentProvider.processPayment(txId, amount).
- registry.register(cryptoAdapter) — zero existing files modified.
- registry.getProvider("Crypto").processPayment("TX-001", 99.99) succeeds via the adapter.

Expected Console Output :

[ADAPTER] Registered providers: CreditCard, UPI, Wallet

[ADAPTER] CryptoPaymentAdapter registered at runtime.

[ADAPTER] Crypto payment result: true

[ADAPTER] Incompatible FakeCryptoAPI.sendCrypto() hidden behind IPaymentProvider.

6.3 Scenario - C : Nested Bundle Inventory Propagation

- Patterns demonstrated: Composite, Proxy, Iterator
- Create leaf products: bandages (stock 50), antiseptic (30), paracetamol (20).
- Create medKit bundle with 10% discount containing all three.
- Create emergencyKit containing medKit (2-level nesting, 3 items deep).
- emergencyKit.isAvailable() returns true before depletion.
- Deplete paracetamol stock to 0 via SecureInventoryProxy.
- emergencyKit.isAvailable() now returns false — propagated up two nesting levels.

Expected Console Output

[COMPOSITE] bandages.isAvailable() = true

[COMPOSITE] medKit.isAvailable() = true (before depletion)

[COMPOSITE] emergencyKit.isAvailable() = true (before depletion)

[COMPOSITE] Depleting paracetamol stock...

[COMPOSITE] paracetamol.isAvailable() = false

[COMPOSITE] medKit.isAvailable() = false (propagated from child)

[COMPOSITE] emergencyKit.isAvailable() = false (propagated from grandchild)

7. Conclusion :

Aura Retail OS overcomes the limitations of previous monolithic kiosk systems by offering a modular, scalable, and extensible smart-city retail platform designed with Object-Oriented principles and design patterns. It supports various kiosk settings such as hospitals, metro stations, campuses, and disaster relief zones, ensuring low coupling and high adaptability. The project effectively showcases core OOP concepts like Encapsulation, Abstraction, Inheritance, and Polymorphism through a subsystem-based architecture and interface-driven communication. Key features such as dynamic hardware replacement, secure inventory management, runtime payment integration, and event-driven monitoring are implemented using patterns like Bridge, Decorator, Adapter, Composite, Proxy, Observer, and Command.

Moreover, the system enforces real-world constraints including emergency purchase limits, prescription verification, hardware dependency checks, and atomic transaction management to guarantee reliability and consistency. The React-based frontend dashboard and simulation scenarios also tested the platform's scalability and extensibility. Overall, Aura Retail OS effectively showcases an enterprise-level modular architecture that is well-suited for future smart-city retail infrastructure.